



CODEx COLLABORATION STANDARD

MePonto Codex Team Collaboration Manual v1.0

三名开发人员使用 Codex 同步开发不同模块的统一执行手册。

Brand	MePonto
System	PontoSys
Scope	Module development, validation, commit, PR, and release workflow
Date	2026-05-26

MePonto Codex Team Collaboration Manual

v1.0

本文档用于规范 MePonto 三名开发人员使用 Codex 同步开发不同模块的方式，确保所有人按照同一套架构规则、模块边界、验证流程和提交规范工作。

1. 目标

MePonto 当前系统包括：

- 主后台
- 加盟商系统
- Leader 系统
- 骑手端 App
- SOP 与培训系统

后续还会增加：

- Partner 后台
- 供应链后台
- 积分商城系统
- 积分商城后台
- 养成宠物系统
- 调度系统
- 风控系统
- 规则引擎

因此协作目标不是简单地让三个人同时写代码，而是建立一套可以长期扩展的平台级开发机制。

核心目标：

所有 Codex 都从仓库规则出发，不从个人习惯出发。

2. 核心原则

1. 仓库规则优先于个人口头习惯。
2. 每个开发人员负责清晰的模块边界。
3. 新模块必须先定义契约，再写代码。

4. 共享代码必须谨慎修改。
5. 所有新能力必须可灰度、可关闭、可回滚。
6. 涉及钱、积分、库存、奖励、结算，必须采用 ledger 思维。
7. 所有界面必须考虑中文、英文、葡语支持。
8. Codex 可以执行开发和验证，但提交必须由开发人员明确触发。
9. `main` 必须保持可部署。
10. `dev` 作为日常集成分支。

语言要求不是文案收尾工作，而是功能验收的一部分。任何新增页面、SOP、导出、报错、通知和培训内容，都必须同时考虑中文、英文、葡语。

3. 项目内规则文件

当前项目已经配置以下文件：

```
AGENTS.md
docs/mepon-to-ecosystem-development-standard-v2.md
docs/mepon-to-ecosystem-os-v2-diagram.md
docs/module-development-playbook.md
docs/module-contract-template.md
docs/pr-checklist.md
docs/codex-team-collaboration-manual.md
.github/PULL_REQUEST_TEMPLATE.md
.github/CODEOWNERS
.github/workflows/codex-ci.yml
scripts/module-guard.mjs
scripts/codex-preflight.mjs
```

各文件作用：

文件	作用
AGENTS.md	Codex 进入项目后必须遵守的总规则
meponto-ecosystem-development-standard-v2.md	平台级系统开发规范
meponto-ecosystem-os-v2-diagram.md	架构图示
module-development-playbook.md	模块开发流程
module-contract-template.md	新模块接入模板
pr-checklist.md	PR 合并前检查清单
PULL_REQUEST_TEMPLATE.md	GitHub PR 模板
CODEOWNERS	核心代码负责人规则
codex-ci.yml	GitHub 自动检查
module-guard.mjs	模块规则检查
codex-preflight.mjs	Codex 提交前检查

4. 三人分工建议

开发人员	主要负责模块	常规修改范围
开发人员 A	加盟商、财务、合作方案	app/franchise , app/finance , franchise docs
开发人员 B	骑手、Leader、移动端	app/riders , app/leaders , app/mobile
开发人员 C	商城、积分、供应链、宠物系统	future marketplace, rewards, supply-chain, gamification modules

共享代码需要 review：

```

app/lib
app/api
app/components
package.json
docs/meponto-ecosystem-development-standard-v2.md
scripts
.github

```

5. 分支策略

建议分支结构：

```
main
  生产可部署分支

dev
  日常集成分支

codex/<module>--<task>
  单个开发任务分支
```

示例：

```
codex/franchise-settlement
codex/rider-onboarding
codex/marketplace-catalog
codex/supply-chain-inventory
codex/gamification-pet-level
```

合并顺序：

```
codex/* -> dev -> main
```

6. 每个开发人员的初始操作

每个开发人员第一次开始工作时：

```
cd "/Users/ishak/Documents/New project"
git checkout dev
git pull origin dev
```

创建自己的任务分支：

```
git checkout -b codex/<module>--<task>
```

例如：

```
git checkout -b codex/rider-onboarding
```

7. 给 Codex 的标准启动指令

每个开发人员开始任务前，建议先对 Codex 说：

请先读取 AGENTS.md，并严格遵守 docs/module-development-playbook.md、docs/module-contract-template.md、docs/pr-checklist.md 的规则。

本次只开发 <模块名> 模块。

不要修改 auth、ledger、module registry、integration gateway、shared api，除非你先说明原因并获得确认。

完成后运行 `npm run codex:preflight`。

示例：

请先读取 AGENTS.md，并严格遵守项目规则。

本次只开发 marketplace catalog 模块。

不要修改 rider、franchise、auth、ledger、module registry。

完成后运行 `npm run codex:preflight`。

8. 新模块开发流程

新增模块时，不能直接开始写页面。必须先完成模块契约。

流程：

1. 复制 `docs/module-contract-template.md`。
2. 填写模块名称、负责人、状态、路由、feature flag。
3. 定义允许角色和权限。
4. 定义模块私有数据。
5. 定义模块对外 API。
6. 定义 inbound / outbound events。
7. 判断是否涉及 ledger。
8. 判断是否涉及 rule engine。
9. 判断是否需要 read model。
10. 定义中英葡语言支持。
11. 定义灰度和回滚方式。
12. 再进入代码开发。

新增模块默认状态：

`disabled` 或 `beta`

不得一开始直接 `active`。

9. 模块接入总系统的逻辑

新模块不能直接进入系统主流程，必须经过以下控制点：

```
Module Contract
  -> Module Registry
  -> Feature Flag
  -> RBAC / Scope
  -> Integration Gateway
  -> Event / API / Read Model
  -> Monitoring
  -> Beta
  -> Active
```

也就是说，新模块是否允许接入总系统，应该由以下能力共同控制：

- Module Registry：控制模块是否存在、是否启用
- Feature Flag：控制灰度、开关、范围
- RBAC：控制谁能访问
- Integration Gateway：控制模块之间如何通信
- Event Schema：控制事件格式
- Read Model：控制跨模块数据展示
- Audit：记录敏感操作

10. Codex 可以直接做的事情

这些任务可以放心交给 Codex 执行和验证：

普通页面开发
表格 / 表单 / dashboard
模块 UI
API route 基础实现
mock data
seed data
SOP 页面
培训文档
中英葡翻译补全
模块契约草稿
PR 描述
本地检查脚本
smoke test
构建验证
文档整理
页面截图检查
导出 HTML / PDF 草稿

11. 需要人确认后再交给 Codex 的事情

以下任务必须先由负责人确认方向，再让 Codex 执行：

登录系统
RBAC 权限模型
数据库结构
钱、积分、库存、奖励、结算
ledger
Module Registry
Integration Gateway
事件总线
调度核心
风控核心
PIX
WhatsApp 正式集成
第三方平台 API
生产部署配置
合同
加盟政策
品牌规则
定价规则
KPI 考核规则

11A. 语言支持要求

MePonto / PontoSys 的系统语言要求：

中文 zh
英文 en
葡语 pt

使用原则：

1. 中文用于内部业务确认、政策讨论、总部管理和文档初稿。
2. 葡语用于巴西本地骑手、站点、加盟商、培训、SOP 和运营场景。
3. 英文用于技术交付、后台标签、跨团队协作和通用管理界面。
4. 品牌名统一使用 MePonto。
5. 系统名统一使用 PontoSys。
6. 不允许在同一个用户可见标签里随意混用三种语言。
7. 技术标识、API 字段、事件名可以使用英文。
8. 如果某次任务只做单语言，PR 里必须说明原因，并列出的语言版本。

必须检查语言覆盖的范围：

导航菜单
页面标题
模块标题
按钮
表单标签
输入提示
筛选项
表格标题
状态标签
空状态
加载状态
错误状态
成功提示
API 返回给用户看的错误信息
SOP 内容
培训内容
PDF / HTML 导出内容
WhatsApp 消息模板
通知模板
加盟商方案
骑手端 App 文案
Leader 工作台文案

Codex 开发页面或内容时，必须在完成前自查：

是否有中文
是否有英文
是否有葡语
是否有遗漏按钮/表格/报错/空状态
是否仍有旧品牌或旧平台名称
是否正确使用 MePonto 和 PontoSys

语言问题属于 PR 阻断项。除非负责人明确批准单语言临时发布，否则不能合并。

12. 提交前验证流程

普通模块开发完成后运行：

```
npm run codex:preflight
```

该命令会执行：

```
module guard
production build
```

高风险变更或上线前运行：

```
npm run codex:preflight:full
```

该命令会执行：

```
module guard
production build
full smoke check
accessibility smoke
workflow smoke
```

当前高风险变更包括：

```
app/lib
app/api
app/components
auth
RBAC
finance
ledger
rewards
points
inventory
settlement
Module Registry
Integration Gateway
events
realtime
rule engine
deployment
```

13. Codex 提交流程

Codex 不应该在没有开发人员明确要求时自动提交。

推荐提交指令：

```
运行 npm run codex:preflight。  
如果通过，只 stage 本次任务相关文件，并创建 commit。  
commit message 使用: <module>: <change summary>
```

示例：

```
运行 npm run codex:preflight。  
如果通过，只 stage marketplace catalog 相关文件，并创建 commit。  
commit message: marketplace: add catalog module beta page
```

高风险提交：

```
运行 npm run codex:preflight:full。  
通过后再提交。
```

14. PR 流程

提交后创建 PR：

```
codex/<module>-<task> -> dev
```

PR 必须说明：

- 本次改了哪个模块
- 是否改了共享代码
- 是否涉及权限
- 是否涉及钱、积分、库存、奖励、结算
- 是否新增事件
- 是否新增 API
- 是否支持中英葡
- 是否有单语言例外说明
- 已经运行哪些检查

PR 合并前必须完成：

```
npm run codex:preflight
```

合并到 `main` 前必须完成：

```
npm run codex:preflight:full
```

15. GitHub 自动检查

项目已经配置：

```
.github/workflows/codex-ci.yml
```

当代码 push 或 PR 到以下分支时会自动运行：

```
dev  
main
```

自动检查包括：

```
npm ci  
npm run module:guard  
npm run build  
npm run check
```

如果 CI 不通过，不允许合并。

16. CODEOWNERS 规则

项目已经配置：

```
.github/CODEOWNERS
```

建议在 GitHub 中开启 branch protection，并要求 CODEOWNERS review。

重点：

- `app/lib` 需要 core review
- `app/api` 需要 core review
- `app/components` 需要 core review
- `app/franchise` 和 `app/finance` 由 franchise owner review
- `app/riders`、`app/leaders`、`app/mobile` 由 rider owner review
- `docs` 和 `app/sops` 由 ops owner review

17. 多人开发时如何避免冲突

规则：

1. 每个人每天开始前先 pull。

2. 每个人只在自己的模块范围内工作。
3. 不要多人同时修改同一个共享文件。
4. 修改共享代码前先说明原因。
5. 小步提交，不要一次提交太多文件。
6. 不要 force push 到别人的分支。
7. 不要用 destructive git 命令解决冲突。
8. 冲突时先判断模块所有权，再决定保留哪边逻辑。

18. 每日工作建议

每天开始：

```
git checkout dev
git pull origin dev
git checkout -b codex/<module>-<task>
```

开发中：

```
npm run codex:preflight
```

准备 PR：

```
git status
git diff
npm run codex:preflight
```

上线前：

```
npm run codex:preflight:full
```

19. 新增商城模块示例

如果新增积分商城模块，执行步骤：

1. 新建模块契约：

```
docs/modules/marketplace-contract.md
```

2. 定义模块：

```
Module name: marketplace
Route: /marketplace
Status: disabled
Feature flag: marketplace.enabled
Owner: Developer C
```

3. 定义权限:

```
Admin: manage_marketplace
Franchise: view_marketplace_orders
Rider: use_marketplace
```

4. 定义数据边界:

```
marketplace owns catalog, orders, redemption requests.
marketplace reads rider profile and points balance through API/read model.
marketplace cannot directly modify rider balance.
marketplace must follow docs/mepon-to-points-economy-standard.md.
```

5. 定义事件:

```
marketplace.order.created.v1
marketplace.order.cancelled.v1
marketplace.redemption.completed.v1
```

6. 定义 ledger:

```
points debit
points refund
inventory reservation
inventory release
```

7. 开发 beta 页面和 API。

8. 运行:

```
npm run codex:preflight
```

9. 创建 PR 到 `dev`。

10. 内部测试后启用 beta。

11. 通过 full check 后合并到 `main`。

20. 推荐给 Codex 的常用指令

开发模块：

请读取 AGENTS.md。本次只开发 <module> 模块，保持模块边界，不要修改共享核心代码。完成后运行 `npm run codex:preflight`。

检查模块：

请检查 <module> 模块是否符合 AGENTS.md、module-development-playbook 和 module-contract-template 的要求，列出缺口并修复低风险问题。

提交代码：

运行 `npm run codex:preflight`。通过后，只 stage 本次任务相关文件，并创建 commit。不要 stage 无关文件。

高风险检查：

这次涉及权限/API/财务，请运行 `npm run codex:preflight:full`，并汇总所有失败或 warning。

新增模块：

基于 docs/module-contract-template.md 为 <module> 创建模块契约，然后实现最小 beta 版本。必须包含 feature flag、权限说明、API、事件和验证步骤。

21. 最终执行标准

一个模块可以合并，必须满足：

模块契约清楚
模块边界清楚
权限清楚
数据边界清楚
事件版本清楚
语言支持清楚
语言例外已说明或不存在
feature flag 已配置
preflight 通过
PR checklist 完成
CI 通过
负责人 review 通过

一句话总结：

Codex 负责高效执行和验证，人负责边界、策略、商业规则和最终合并判断。